Homework — 2.1.5 Thinking Concurrently — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. (a) [2] 1 mark: each frame is computed from its own data and does **not** depend on the result of any other frame — there is no data dependency between them. 1 mark: so they can be distributed and processed **simultaneously/in parallel** without one having to wait for another, then collected together.

(b) [1] Any one: a multi-core processor / multiple cores / multiple processors / a GPU / a render farm of several machines — to run frames truly in parallel.

2. [2] 1 mark: the spell-check runs as a separate task in the background. 1 mark: so the **user interface stays responsive** — the user can keep typing/editing instead of waiting for the (long) spell-check to finish, improving experience.

3. (a) [1] A **race condition**.

(b) [2] 1 mark: both processes accessed the **same shared data** (the stock count) and each read "1" before either had written its update. 1 mark: because the updates were not coordinated/synchronised, the final result depended on the unpredictable **timing/order** of the two operations, allowing the same item to be sold twice.

4. [2] Any two (1 each): on a **single core** there is no true parallelism — the CPU only **time-slices**, so no real speed-up; there is **overhead** in creating, managing and synchronising/recombining threads; some problems are **inherently sequential** because later steps depend on earlier outputs, so they cannot be parallelised.

5. [4] Up to 4: tasks (b) reserve flight and (c) reserve hotel are **independent** of each other, so they could run **concurrently** (1). A sensible dependency: (a) charging the card and/or both reservations should succeed **before** (d) the confirmation email is sent, since the email reports the result — so (d) must be **sequenced after** the others (1). Benefit (1): doing the independent reservations at once makes the booking complete faster / keeps the site responsive. Trade-off (1): concurrency is harder to design and debug, risks **race conditions** if tasks touch shared data (e.g. the same last seat), and results must be **synchronised/recombined** before confirming; on limited hardware it may not be truly faster. Must apply to the booking scenario and weigh both sides.

Part B (6 marks)

6. [2] 1 mark + 1 mark: `IF movesLeft = 0 OR score >= 100` (accept equivalent variable names and `score = 100` only if "exactly" — here `>= 100` for "scored 100"). Must use the **OR** operator and be a valid Boolean.

7. [2] 1 mark for the first three rows correct, 1 mark for all four correct:

A	B	Q
0	0	0
0	1	0
1	0	1
1	1	0

8. [2] 1 mark + 1 mark for a clear difference: a **static** structure has a **fixed size** set at compile/creation time and uses contiguous memory (e.g. an array) **[1]**; a **dynamic** structure can **grow and shrink** at run time, allocating/freeing memory as needed (e.g. a linked list) **[1]**.

Total: 14 + 6 = 20 marks